

ChatGPT Usage Log

 **time_py.txt**
Document

write a python script to find the following things about a set of data such as the txt file attached: Minimum value, first percentile, 10th percentile, mean, median, 90th percentile, 99th percentile and maximum value

Here's a simple Python script that reads a text file containing one numeric value per line (like your uploaded file) and calculates:

- Minimum value
- 1st percentile
- 10th percentile
- Mean
- Median
- 90th percentile
- 99th percentile
- Maximum value

The uploaded file appears to be a list of numbers, one per line.  time.py

 Python



 Run

```
import numpy as np

# Path to your data file
filename = "time_py.txt"

# Read numbers from file
with open(filename, "r") as f:
    data = np.array([float(line.strip()) for line in f if line.strip()])

# Calculate statistics
stats = {
    "Minimum": np.min(data),
    "1st Percentile": np.percentile(data, 1),
    "10th Percentile": np.percentile(data, 10),
    "Mean": np.mean(data),
    "Median": np.median(data),
    "90th Percentile": np.percentile(data, 90),
    "99th Percentile": np.percentile(data, 99),
    "Maximum": np.max(data)
}

# Print results
print("\nStatistics:")
for name, value in stats.items():
    print(f"{name:15}: {value:.3f}")
```

For learning purposes, you could even add:

 Python



 Run

```
print("Number of data points =", len(data))
```

before the statistics section so you know how many values were read from the file. This is often a useful sanity check when working with experimental or timing data.

     ...

```
# Display results

print()
print("Statistics")
print("-----")

print("Minimum value      =", minimum_value)

print("1st percentile     =", first_percentile)

print("10th percentile     =", tenth_percentile)

print("Mean                =", mean_value)

print("Median              =", median_value)

print("90th percentile      =", ninetyeth_percentile)

print("99th percentile      =", ninety_ninth_percentile)

print("Maximum value       =", maximum_value)
```

If you want to see exactly what each NumPy function is doing conceptually:

Python



Run

```
minimum_value = np.min(data)
```

finds the smallest number in the array.

Python



Run

```
mean_value = np.mean(data)
```

adds all the numbers together and divides by the number of values.

Python



Run

```
median_value = np.median(data)
```

sorts the data and finds the middle value.

Python



Run

```
first_percentile = np.percentile(data, 1)
```

sorts the data and finds the value below which 1% of the observations lie.

now can you please do the same things for calculating and displaying the statistics

Certainly. Instead of putting all the calculations into a dictionary, you can calculate each statistic individually so that every step is visible.

```
<> Python Run

import numpy as np

filename = "time_py.txt"

# Read data from file
data = []

with open(filename, "r") as f:

    for line in f:

        cleaned_line = line.strip()

        if cleaned_line != "":

            value = float(cleaned_line)

            data.append(value)

# Convert list to NumPy array
data = np.array(data)

# Calculate statistics one at a time

minimum_value = np.min(data)

first_percentile = np.percentile(data, 1)

tenth_percentile = np.percentile(data, 10)

mean_value = np.mean(data)

median_value = np.median(data)

ninetieth_percentile = np.percentile(data, 90)

ninety_ninth_percentile = np.percentile(data, 99)

maximum_value = np.max(data)

data = np.array(data)

print("Finished reading file")
print("Number of values:", len(data))
```

This is closer to how you would write it while learning Python, before gradually replacing parts with more compact constructs like list comprehensions.

📄 🌟 🗣️ ⬆️ ↺ ⋮

the loop does:

Python



Run

```
line = "3433899\n"
cleaned_line = "3433899"
value = 3433899.0
data = [3433899.0]

line = "2579\n"
cleaned_line = "2579"
value = 2579.0
data = [3433899.0, 2579.0]

line = "1307\n"
cleaned_line = "1307"
value = 1307.0
data = [3433899.0, 2579.0, 1307.0]
```

and so on until the entire file has been read.

You can make the process even more explicit with debug prints:

Python



Run

```
import numpy as np

filename = "time_py.txt"

data = []

with open(filename, "r") as f:

    for line in f:

        print("Raw line:", repr(line))

        cleaned_line = line.strip()
        print("After strip:", cleaned_line)

        if cleaned_line != "":

            value = float(cleaned_line)
            print("Converted to number:", value)

            data.append(value)
            print("Current list length:", len(data))
            print()
```

can you please spread those 2 lines out over more lines to clear show each step

Yes. The original code compresses several operations into a single line. Here's the same logic written out step-by-step:

```
<> Python Run

import numpy as np

filename = "time_py.txt"

# Create an empty list to store the numbers
data = []

# Open the file
with open(filename, "r") as f:

    # Read each line in the file
    for line in f:

        # Remove spaces and newline characters
        cleaned_line = line.strip()

        # Ignore blank lines
        if cleaned_line != "":

            # Convert text to a number
            value = float(cleaned_line)

            # Add the number to our list
            data.append(value)

# Convert the Python list into a NumPy array
data = np.array(data)

print(data)
```

For example, if the file contains:

```
3433899
2579
1307
519
370
```

NumPy arrays are useful because functions like:

```
</> Python   Run  
np.mean(data)  
np.median(data)  
np.percentile(data, 90)
```

operate directly on them.

If you're new to Python, I'd recommend temporarily expanding the code into the longer version below and printing intermediate values:

```
</> Python   Run  
import numpy as np  
  
filename = "time_py.txt"  
  
data = []  
  
with open(filename, "r") as f:  
    for line in f:  
        print("Raw line:", repr(line))  
  
        cleaned = line.strip()  
        print("After strip:", cleaned)  
  
        if cleaned:  
            value = float(cleaned)  
            print("Numeric value:", value)  
  
            data.append(value)  
  
data = np.array(data)  
  
print(data[:10])
```

Running this once makes it very clear how the text file is being converted into numerical data.

     ...

The list comprehension:

Python



Run

```
[float(line.strip()) for line in f if line.strip()]
```

means:

Python



Run

```
data = []

for line in f:
    if line.strip():          # skip blank lines
        data.append(float(line.strip()))
```

So after reading the example file:

Python



Run

```
data = [
    3433899.0,
    2579.0,
    1307.0,
    519.0,
    370.0
]
```

Finally:

Python



Run

```
np.array(data)
```

converts the Python list into a NumPy array:

Python



Run

```
array([
    3433899.,
    2579.,
    1307.,
    519.,
    370.
])
```

Python reads one line at a time:

Python

```
line = "3433899\n"  
line = "2579\n"  
line = "1307\n"  
...
```



Run

Notice the `\n` at the end of each line. This is the newline character.

That's why we use:

Python

```
line.strip()
```



Run

For example:

Python

```
"3433899\n".strip()
```



Run

becomes

Python

```
"3433899"
```



Run

Next:

Python

```
float(line.strip())
```



Run

converts the text into a number:

Python

```
"3433899" -> 3433899.0  
"2579" -> 2579.0
```



Run

going back to the original code, can you please explain how you extract the dat from the txt file

Certainly. The key line is:

```
Python
with open(filename, "r") as f:
    data = np.array([float(line.strip()) for line in f if line.strip()])
```

Let's break it down.

First, the file is opened:

```
Python
with open(filename, "r") as f:
```

- `open()` opens the file.
- `"r"` means "read mode".
- `f` becomes a file object.
- `with` automatically closes the file when you're done.

Suppose `time_py.txt` contains:

```
3433899
2579
1307
519
370
```

Then `f` behaves like a sequence of lines.

When you do:

```
Python
for line in f:
```

A robust solution is to make the file path relative to the script rather than the working directory:

```
Python
from pathlib import Path

script_dir = Path(__file__).parent
filename = script_dir / "time_py.txt"

with open(filename, "r") as f:
    data = [float(line.strip()) for line in f]
```

If you're using VS Code, also tell me:

1. How you opened VS Code (Open Folder or just Open File)?
2. What appears after running `print(os.getcwd())`?
3. The exact filename as shown in Finder (including capitalization and extension).

Those three pieces of information will identify the problem quickly.

📄 📁 🔍 ↕️ ↻️ ⋮

I still get this error when the text file is in the same folder as the python file "FileNotFoundError: [Errno 2] No such file or directory: 'time_py.txt'"

This usually means Python is running from a different directory than the one containing the file.

Add these lines at the top of your script:

```
<> Python Run  
  
import os  
  
print("Current working directory:")  
print(os.getcwd())  
  
print("\nFiles in this directory:")  
print(os.listdir())
```

Then run the script again.

You'll see something like:

```
Current working directory:  
/Users/ben/some/folder  
  
Files in this directory:  
['analysis.py']
```

If `time_py.txt` is not listed, Python is looking in the wrong folder.

Another useful check is to print the location of the script itself:

```
<> Python Run  
  
import os  
  
print("Script location:")  
print(os.path.dirname(os.path.abspath(__file__)))
```

If you're unsure where VS Code is running from, add:

```
<> Python Run  
  
import os  
print(os.getcwd())
```

before opening the file. It will print the directory Python is searching in.

If you tell me:

- macOS or Windows?
- where your `.py` file is located?
- where your `.txt` file is located?

I can tell you the exact path to use.

where must the text file be for the program to see it?

It depends on how you specify the filename.

In your script, you had:

```
<> Python
filename = "time_py.txt"
```

Run

This is a **relative path**, which means Python looks for `time_py.txt` in the **current working directory** (usually the folder you opened in VS Code).

The easiest approach is:

1. Create a folder, e.g. `PythonProject`.
2. Put both:
 - `analysis.py`
 - `time_py.txt`in that folder.
3. In VS Code, open that folder via **File → Open Folder**.
4. Run `analysis.py`.

Your folder should look like:

```
PythonProject/
├── analysis.py
└── time_py.txt
```

Run

If the file is elsewhere, you can give the full path instead:

```
<> Python
filename = "/Users/yourname/Documents/time_py.txt"
```

Run

or on Windows:

```
<> Python
filename = r"C:\Users\yourname\Documents\time_py.txt"
```

Run

If you prefer not to depend on NumPy, here's a pure-Python version:

```
Python Run

from statistics import mean, median

filename = "time_py.txt"

with open(filename, "r") as f:
    data = sorted(float(line.strip()) for line in f if line.strip())

def percentile(data, p):
    n = len(data)
    k = (n - 1) * p / 100
    f = int(k)
    c = min(f + 1, n - 1)

    if f == c:
        return data[f]

    return data[f] + (data[c] - data[f]) * (k - f)

print(f"Minimum      : {min(data):.3f}")
print(f"1st Percentile : {percentile(data, 1):.3f}")
print(f"10th Percentile : {percentile(data, 10):.3f}")
print(f"Mean           : {mean(data):.3f}")
print(f"Median          : {median(data):.3f}")
print(f"90th Percentile : {percentile(data, 90):.3f}")
print(f"99th Percentile : {percentile(data, 99):.3f}")
print(f"Maximum        : {max(data):.3f}")
```

If your file may contain headers, timestamps, or multiple columns, I can adapt the script accordingly.

     ...  Sources

	ONSConstID	ConstituencyName	RegNationID	RegNationName	NatC
1	E14001063	Aldershot	E12000008	South East	Engl
2	E14001063	Aldershot	E12000008	South East	Engl
3	E14001063	Aldershot	E12000008	South East	Engl
4	E14001064	Aldridge-Brownhill s	E12000005	West Midlands	Engl

I have the following spreadsheet. There are currently 3 rows corresponding to each constituency. I only care about the 3rd row, the number of people without disabilities and so need to reformat the data such that each constituency corresponds to 1 row. What are the different ways to do this that aren't doing it manually?



Stopped thinking >

Looking at your spreadsheet, the data are already in a tidy repeating pattern: for each constituency there are three rows:

1. Day-to-day activities limited a lot
2. Day-to-day activities limited a little
3. Not disabled under the Equality Act

If you only want the third row for each constituency, there are several non-manual approaches.

1. Filter on the "Not disabled" category (recommended)

Since the third row is explicitly identified by the `variables` or `groups` column, you can simply filter those rows.

In Excel:

- Select the data.
- Turn on Filters (`Data → Filter`).
- Filter the `variables` column to:
 - `Not disabled under the Equality Act`
- Copy the filtered results to a new sheet if needed.

Or with Power Query:

- `Data → From Table/Range`
- Filter `variables` to `"Not disabled under the Equality Act"`
- Load back to Excel.

This is the most robust method because it does not rely on the row order always being the same.

2. Use Power Query to keep every third row

If the third-row pattern is guaranteed:

1. Load the table into Power Query.
2. Add an Index column starting from 0.
3. Filter the Index column using:

```
Number.Mod([Index],3)=2
```



4. Remove the Index column and load the result.

This keeps rows 2, 5, 8, 11, etc.

3. Use an Excel formula to extract every third row

Suppose your data start in row 2.

You can create a new table using:

```
<> excel
=FILTER(A2:K10000,MOD(ROW(A2:A10000)-ROW(A2),3)=2)
```



This dynamically returns every third row.

4. Use a PivotTable

If all you need is one record per constituency:

- Insert a PivotTable.
- Put `ConstituencyName` in Rows.
- Filter `variables` to `Not disabled under the Equality Act`.
- Add the fields you want (`Con_num`, `Con_pc`, etc.) to Values.

This gives one row per constituency without modifying the source data.

5. Use Python/Pandas

For larger workflows:

```
<> Python
import pandas as pd

df = pd.read_excel("Disabilities Index Calculation.xlsx")

result = df[
    df["variables"] == "Not disabled under the Equality Act"
]

result.to_excel("disability_cleaned.xlsx", index=False)
```



▶ Run

Or simply:

```
<> Python
result = df.iloc[2::3]
```



▶ Run

if the third-row pattern is guaranteed.



For this particular spreadsheet, filtering on:

```
variables = "Not disabled under the Equality Act"
```



is the safest approach because it will continue to work even if future datasets contain missing rows or a different ordering.

     ...

Unable to display this message due to an error.

Stopped thinking >

Assuming your Excel file contains the columns:

- digital_equity_index
- socio_econ_index
- RegNationName

the following MATLAB script will read the spreadsheet, create a scatter plot of digital_equity_index against socio_econ_index, and assign a different colour to each RegNationName group.

« MATLAB



```
%% Read data from Excel
filename = 'Digital Equity Index Calculation.xlsx';

T = readtable(filename, ...
    'Sheet', 'Digital Equity Index Values');

%% Get unique nations/regions
groups = unique(T.RegNationName);

%% Create figure
figure;
hold on;

% Generate distinct colours
colours = lines(numel(groups));

%% Plot each group separately
for i = 1:numel(groups)

    idx = strcmp(T.RegNationName, groups{i});

    scatter( ...
        T.socio_econ_index(idx), ...
        T.digital_equity_index(idx), ...
        40, ...
        colours(i,:), ...
        'filled', ...
        'DisplayName', groups{i});

end

%% Formatting
xlabel('Socio-Economic Index');
ylabel('Digital Equity Index');
title('Digital Equity Index vs Socio-Economic Index');
legend('Location','best');
grid on;
box on;

hold off;
```

If you would like the points labelled with constituency names (parl_const_name) as well, you can add:

« MATLAB



```
text( ...
    T.socio_econ_index, ...
    T.digital_equity_index, ...
    T.parl_const_name, ...
    'FontSize', 7);
```

after the scatter plotting loop. This will place a label next to every point.

📄 🌟 🔄 ⬆️ ↺ ...



I can't seem to make it work with an excelspreadsheet as an input, what format could I convert the spreadsheet into to make it easier for matlab to read and plot?

CSV is usually the easiest format for MATLAB to work with. Excel files often cause problems because of:

- Multiple sheets
- Spaces or special characters in column names
- Mixed text and numeric data
- Hidden rows/columns
- Different Excel versions

Try saving the relevant sheet as a CSV file:

Excel → File → Save As → CSV (Comma delimited) (*.csv)

Then in MATLAB:

```
«» MATLAB
T = readtable('digital_equity_data.csv');
```

You can check the column names MATLAB has imported with:

```
«» MATLAB
T.Properties.VariableNames
```

and then plot using the exact names MATLAB has assigned.

For example:

```
«» MATLAB
groups = unique(T.RegNationName);

figure;
hold on

colours = lines(numel(groups));

for i = 1:numel(groups)
    idx = strcmp(T.RegNationName, groups{i});

    scatter(T.socio_econ_index(idx), ...
            T.digital_equity_index(idx), ...
            50, colours(i,:), 'filled');
end

legend(groups, 'Location', 'bestoutside')
xlabel('Socio-Economic Index')
ylabel('Digital Equity Index')
grid on
hold off
```

Another very MATLAB-friendly option is to save the data as a `.mat` file:

```
«» MATLAB
T = readtable('digital_equity_data.csv');
save('digital_equity_data.mat', 'T')
```

and later simply load:

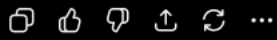
and later simply load:

«» MATLAB



```
load('digital_equity_data.mat')
```

If you're still having trouble, the issue may actually be the sheet name or column names rather than the Excel format itself. If you upload the spreadsheet, I can inspect the structure and give you the exact `readtable` command and plotting code for that file.



	parl_const_name	RegNationName	broadband_index	mobile_index	socio_e
1	Aberafan Maesteg	Wales	79.15	62.27315	28.1328
2	Aberdeen North	Scotland	81.18	89.3678	44.232
3	Aberdeen South	Scotland	78.065	84.9662	62.5014
4	Aberdeenshire North and Moray East	Scotland	48.075	53.19585	39.7773
5	Airdrie and Shotts	Scotland	76.38	56.47545	30.0619
6	Aldershot	South East	78.66	59.4116	55.792

I now have the above csv file. Please write a MATLAB program to plot the digital equity index against the socioeconomic index, assigning a different colour to each region, i.e each different word in the RegNationName column

Stopped thinking >

I checked the file structure. The relevant columns are:

- digital_equity_index
- socio_econ_index
- RegNationName

The following MATLAB script will read your CSV file and plot Digital Equity Index against Socio-Economic Index, using a different colour for each region in RegNationName .

MATLAB

```
%% Read data
T = readtable('digital_equity_values(2).csv');

%% Get unique regions
regions = unique(T.RegNationName);

%% Create figure
figure;
hold on;

% Generate distinct colours
colours = lines(length(regions));

%% Plot each region
for i = 1:length(regions)
    idx = strcmp(T.RegNationName, regions{i});

    scatter( ...
        T.socio_econ_index(idx), ...
        T.digital_equity_index(idx), ...
        50, ...
        colours(i,:), ...
        'filled', ...
        'DisplayName', regions{i});
end

%% Format plot
xlabel('Socio-Economic Index');
ylabel('Digital Equity Index');
title('Digital Equity Index vs Socio-Economic Index');

legend('Location','best');
grid on;
box on;

hold off;
```

If you want MATLAB to automatically fit and display a regression line for all points:

» MATLAB



```
p = polyfit(T.socio_econ_index, T.digital_equity_index, 1);  
xfit = linspace(min(T.socio_econ_index), ...  
                max(T.socio_econ_index), 100);  
yfit = polyval(p, xfit);  
plot(xfit, yfit, 'k-', 'LineWidth', 2, ...  
      'DisplayName', 'Linear Fit');
```

Place this immediately before the `legend` command.

Since your dataset contains only a few regions (England, Scotland, Wales and Northern Ireland), the `lines` colormap will produce four clearly distinguishable colours automatically.

